



UNIWERSYTET VIZJA

Zaawansowane zagadnienia algorytmiki i programowania Projekt

Algorytmy w teorii liczb i kryptografia

**Projekt dotyczy implementacji algorytmów arytmetyki modularnej
i teorii liczb w kontekście kryptografii (RSA)**

Prowadzący

dr hab. inż. Agnieszka Duraj

Studenci

Jakub Peciak Album nr: 65848

Paweł Cabaj Album nr 65456

Warszawa 2025

Raport 2

Implementacja bazowa: zaimplementowanie szybkiego potęgowania i algorytmu Euklidesa

Spis treści

1. Informacje wstępne
2. Budowa blokowa aplikacji
3. Implementacja serwera
4. Implementacja klienta
5. Podsumowanie

1. Informacje wstępne

Projekt składa się z dwóch aplikacji łączących się w relacji serwer – klient.

Przygotowany kod wprowadza implementację modelu klient – serwer, który opiera się o kod języka Python. Programy są tak przygotowane, aby pokazać przykład wymiany danych bez szyfrowania oraz z szyfrowaniem wynegocjowanym kluczem publicznym.

Serwer ma za zadanie włączyć nasłuchiwanie na konkretnym adresie IP – tu wybrałem localhost (127.0.0.1) oraz określonym (wolnym) porcie TCP – tu wybrałem (8888). Aplikacja serwera generalnie działa w osobnym wątku (listen_to) a rozruch serwera następuje funkcją (startuj). Pozostałe metody służą do generowania prywatnego i publicznego rozwiązania (protokołu) RSA.

Klient (podobnie) opiera się na własnym wątku (take_of) i jest inicjowany funkcją startową (laduj). Łączy się z serwerem również na określonym gnieździe (socket) ustalonym tak, aby umożliwić komunikację. Funkcja (prośba) odpowiada za pobranie klucza a funkcja (encrypt) za szyfrowanie wiadomości.

Na zrzutach ekranowych pochodzących z edytora kodu pochodzące z IDE przedstawię rysunki, które określają strukturę obu aplikacji. Treść kodu odpowiada na merytorykę problemu generowania kluczy RSA opisaną w pierwszym sprawozdaniu. W kolejnych odsłonach przedstawię strukturę oraz budowę kodu.

W funkcjach main w obu przypadkach ustalony jest jako parametr tzw. socket, który musi być zgodny. A także jak wspomniałem, port musi być wolny.

Programy działają w oparciu o model przedstawiony w pierwszym sprawozdaniu. Tu zajmę się tłumaczeniem fragmentów kodu.

Wcześniej jednak przedstawię scenariusz, jaki posłuży do sprawdzenia poprawności działania aplikacji. Nie mają one powłoki graficznej, więc proponuję otworzyć je w środowisku programistycznym.

- 1) Uruchamiamy serwer
- 2) Uruchamiamy klienta
- 3) Od teraz można wymieniać nieszyfrowane komunikaty między konsolami
- 4) Wpisujemy tekst: „rsa” i potwierdzamy
- 5) Otrzymujemy info zwrotne z kluczem publicznym ze strony serwera
- 6) Jeśli wpisujemy tekst z prefiksem „ENC:” serwer otrzyma szyfrowaną wiadomość

2. Budowa blokowa aplikacji

Zanim przejdę do inspekcji kodu przedstawię tylko budowę blokową obu modułów. Poniżej znajduje się widok struktury (definicji funkcji składowych) kolejno dla serwera oraz klienta. Wszystkie funkcje składowe opisane są w dalszej części sprawozdania. Serwer składa się z definicji funkcji jak na zrzucie. Metody rozpoczynające się na 'generate' służą do obsługi RSA, pozostałe odpowiadają za działanie serwera.

```
1  import socket
2  import threading
3  import time
4  import random
5  import json
6  from math import gcd
7
8  global_n = None
9  global_d = None
10
11
12  # Nasłuchiwanie zdarzeń i przesyłanie klucza RSA + obsługa błędów
13 > def listen_to(client): ...
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57
58
59
60
61
62
63
64
65
66
67
68
69
70
71
72
73
74
75
76
77
78  # Uruchomienie serwera
79 > def startuj(HOST, PORT, message): ...
80
81
82
83
84
85
86
87
88
89
90
91
92
93
94
95
96
97
98
99
100
101
102
103
104
105
106
107
108
109
110
111  # Generuj liczbę pierwszą
112 > def generate_prime(max, exclude): ...
113
114
115
116
117
118
119
120
121
122
123
124 |
125 # Oblicz funkcję Eulera
126 > def generate_n_phi(prime_a, prime_b): ...
127
128
129
130
131
132
133 # Wybór wykładnika szyfrowania (e)
134 # Algorytm Euklidesa (prosty) - biblioteka "math"
135 > def generate_e(phi): ...
136
137
138
139
140
141
142
143 # Generowanie wykładnika deszyfrowania (d)
144 # Algorytm Euklidesa rozszerzony
145 > def generate_d(e, phi): ...
146
147
148
149
150
151
152
153
154
155
156
157
158
159
160 # Algorytm RSA - pełna implementacja
161 > def generate_rsa(): ...
162
163
164
165
166
167
168
169
170
171
172
173
174
175
176
177
178
179
180
181 if __name__ == "__main__":
182
183     startuj('127.0.0.1', 8888, "Połączenie otwarte..")
184
```

Tu natomiast mamy widok struktury programu klienta – i definicje funkcji składowych. Są to metody odpowiadające za wysłanie żądania i kodowanie oraz same podtrzymywanie połączenia.

```
1  import socket
2  import threading
3  import json
4
5  global_n = None
6  global_e = None
7
8  # Żądanie/odbiór klucza publicznego
9  > def prosba(c): ...
35
36
37  # Szyfrowanie wiadomości
38  > def encrypt(c, msg): ...
51
52
53  # Uruchomienie klienta
54  > def laduj(HOST, PORT): ...
88
89
90  # Obsługa wątku klienta
91  > def take_of(c): ...
105
106
107
108  > if __name__ == "__main__": ...
```

3. Implementacja serwera

Serwer po uruchomieniu nasłuchuje na określonym gnieździe rozumianym jako para adres IP + nr portu TCP. Tu dokładnie nasłuchiwanie jest prowadzone na localhost (127.0.0.1:8888) i następuje po wywołaniu w funkcji głównej (main) funkcji „startuj()”, która równocześnie uruchamia całą logikę aplikacji.

Funkcja startuj() działa na następujących zasadach. Następuje bindowanie pary (IP:TCP) oraz uruchomienie nasłuchiwanie (konkretnego klienta). Później w nowym wątku uruchamiamy kolejną funkcję „listen_to()”, która służy do dalszego sterowania połączeniem. W bloku try – except uruchamiamy pętlę while, która oczekuje na możliwość przesłania tekstu – jawnym tekstem, bez szyfrowania oraz obsługuje zamykanie klienta i rozłączanie gniazda.

```
78 # Uruchomienie serwera
79 def startuj(HOST, PORT, message):
80
81     # Ustawienie połączenia
82     s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
83     s.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)
84     s.bind((HOST, PORT))
85     s.listen()
86
87     client, addr = s.accept()
88     print(f"Rozpaczynam nasłuchiwanie na : {HOST}:{PORT}")
89     print(f"DO KLIENTA - wiadomość nieszyfrowana: {message}")
90     client.sendall(message.encode())
91
92     t = threading.Thread(target=listen_to, args=(client,))
93     t.daemon = True
94     t.start()
95
96
97     try:
98         while True:
99             msg = input("Napisz wiadomość (lub 'exit' aby zakończyć): ")
100             if msg.lower() == "exit":
101                 break
102             client.sendall(msg.encode())
103     except KeyboardInterrupt:
104         pass
105     finally:
106         print("Zamykanie połączenia...")
107         client.close()
108         s.close()
109
```

Funkcja `listen_to()` jest dosyć rozwinięta. I streszczając odbiera i dekoduje wiadomości oraz jest wrażliwa na szczegółowe wywołania a w szczególności, łączy klienta (`data`) i dekoduje wiadomość (`msg`). Wyłączone są dwa przypadki, kiedy wiadomość ma treść `<GENERUJ_RSA>` i wtedy uruchamia funkcję `generate_rsa()`, która zgodnie z nazwą wyznacza parametry do komunikacji RSA (klucz publiczny i prywatny). I drugi, kiedy wiadomość ma prefix „ENC:” i wtedy traktuje wiadomość jako szyfrowaną i dzieli ją na bajty a następnie próbuje ją odkodować kluczem prywatnym (z obsługą błędów). W komunikacji jest użyty format Json.

```
12 # Nasłuchiwać zdarzeń i przesyłanie klucza RSA + obsługa błędów
13 def listen_to(client):
14     while True:
15         try:
16             data = client.recv(1024)
17             if not data: ...
18
19
20
21
22         try:
23             msg = data.decode().strip()
24             except UnicodeDecodeError: ...
25
26
27             print(f"\n[OD KLIENTA] {msg}")
28
29             # obsługa komendy generowania RSA
30             if msg.upper() == "GENERUJ_RSA":
31                 print("Żądanie od klienta: wygeneruj klucz RSA.")
32                 try:
33                     n, e, d = generate_rsa() # generowanie parametrów RSA
34                 except Exception as err: ...
35
36
37             # wysyłamy tylko klucz publiczny (n, e) – nigdy d
38             resp = {"status": "ok", "n": str(n), "e": str(e)}
39
40             # konwertujemy liczby do stringów żeby uniknąć problemów z JSON (bezpieczne)
41             client.sendall(json.dumps(resp).encode())
42             print("Wysłano klucz publiczny (n,e) do klienta.")
43             continue
44
45             # Obsługa odczytu wiadomości szyfrowanej
46             elif msg.startswith("ENC:"):
47                 print("coś szyfrem")
48
49                 msg = msg[4:]
50
51                 print(msg)
52                 # enc_msg to string otrzymany od klienta, np. "1234567890123456789"
53                 cif = int(msg) # zamiana na liczbę całkowitą
54
55                 # RSA decryption: M = C^D mod N
56                 m = pow(cif, global_d, global_n) # N to modulus, ten sam co w kluczu publicznym
57                 print(m)
58                 # Zamiana liczby z powrotem na tekst
59                 msg_decrypted = m.to_bytes((m.bit_length() + 7) // 8, "big").decode("latin-1")
60                 print("Odszyfrowana wiadomość:", msg_decrypted)
61
62
63             except ConnectionResetError: ...
64             except Exception as e: ...
65
66         client.close()
67
68
```

Funkcja `generate_rsa()` odpowiada za wyznaczenie pary kluczy publiczny - prywatny (parametrów RSA), wg. algorytmu wskazanego we wcześniejszym sprawozdaniu. Dzieje się to poprzez wywołania funkcji zewnętrznych, opisanych poniżej. Wynik działania to zestaw parametrów (n, e, d) i zapisane liczby d i n jako globalne.

```
160 # Algorytm RSA - pełna implementacja
161 def generate_rsa():
162
163     global global_n, global_d
164     max = 900000000000
165     p = generate_prime(max, None)
166     q = generate_prime(max, p)
167     n = p * q
168     phi = (p - 1) * (q - 1)
169     e = generate_e(phi)
170     d = generate_d(e, phi)
171     global_d = d
172     global_n = n
173     # print("Typ e:", type(e))
174     # print("Typ phi:", type(phi))
175     # print(f"{p}, {q}, {n}, {e}, {d}")
176
177     return n, e, d
178
```

Funkcja `generate_prime()` generuje liczbę pierwszą z zakresu od 800 000 000 000 do 'max' z wyłączeniem liczby 'exclude', gdyż w algorytmie losujemy dwie różne. Wyłączone są wszystkie liczby parzyste. Algorytm przeszukuje pod kątem istnienia dzielnika z zakresu od 3 do $\sqrt{a}$, a jeśli znajdzie losuje ponownie. Jako wynik zwraca liczbę pierwszą. Liczba losowana jest dosyć duża, ze względu na fakt, iż determinuje to jak długą wiadomość można zaszyfrować. W programie liczby p i q pozwalają na zakodowanie 10 znaków. Aby to rozszerzyć można przesyłać informację paczkami (czego nie zaimplementowano).


```

111 # Generuj liczbę pierwszą
112 def generate_prime(max, exclude):
113     while True:
114         a = random.randint(800000000000, max)
115         if a == exclude: continue
116         if a % 2 == 0: continue
117         for i in range(3, int(a**0.5)+1, 2):
118             if a % i == 0:
119                 break
120         else:
121             return a
122
123

```

Funkcja generate_n_phi() zwraca parametr n (dla dwóch różnych liczb pierwszych – iloczyn p i q) oraz funkcję Eulera (liczby względnie pierwsze).

```

125 # Oblicz funkcję Eulera
126 def generate_n_phi(prime_a, prime_b):
127     n = prime_a * prime_b
128     phi = (prime_a - 1) * (prime_b - 1)
129     return n, phi
130
131

```

Funkcja generate_e() określa wykładnik publiczny 'e' z zakresu od 3 do $\Phi - 1$ obliczany z użyciem funkcji math.gcd (NWD), która jest wywoływana dopóki nie znajdzie liczby pierwszej.

```

133 # Wybór wykładnika szyfrowania (e)
134 # Algorytm Euklidesa (prosty) - biblioteka "math"
135 def generate_e(phi):
136     while True:
137         e = random.randint(3, phi - 1)
138         if gcd(e, phi) == 1:
139             return e
140
141

```

Funkcja `generate_d()` określa wykładnik prywatny 'd', który jest wyznaczony przez tzw. rozszerzony algorytm Euklidesa. Formalnie jest to odwrotność modularna liczby 'd' względem 'e'. W kodzie zastosowano tu zabezpieczenie prawidłowego wyniku (jeśli NWD większy od 1, zgłoszony jest wyjątek).

```
143 # Generowanie wykładnika deszyfrowania (d)
144 # Algorytm Euklidesa rozszerzony
145 def generate_d(e, phi):
146
147     t, new_t = 0, 1
148     r, new_r = phi, e
149     while new_r != 0:
150         quotient = r // new_r
151         t, new_t = new_t, t - quotient * new_t
152         r, new_r = new_r, r - quotient * new_r
153     if r > 1:
154         raise Exception("Brak odwrotności modularnej")
155     if t < 0:
156         t = t + phi
157     return t
158
```

4. Implementacja klienta

Klient po włączeniu może połączyć się z serwerem , oczywiście na odpowiednim gnieździe (IP + port). Ma możliwość żądania wygenerowania klucza. Prowadzenia komunikacji bez szyfrowej oraz szyfrowanej. Logikę aplikacji uruchamia funkcja (laduj).

Funkcja `laduj()` podobnie jak funkcja `startuj()` dla serwera, nawiązuje połączenie z serwerem na odpowiednim IP i porcie. Następnie uruchamia funkcję `take_of()`, która obsługuje dalszą część komunikacji. Funkcja, o której mowa oczekuje również w pętli na informację wpisaną do konsoli. Obsługuje dwa przypadki. Wpisano text „rsa” oraz wprowadzono tekst z prefiksem „ENC:” Co w efekcie daje odpowiednio: wysłanie żądania do serwera o ustalenie pary kluczy – funkcja `prosba()` oraz w drugim przypadku wysłanie komunikatu szyfrowanego (jeśli wynegocjowano klucze) – `encrypt()`. Jeśli wpisano „exit” połączenie zostaje zamknięte (`close`).

```
53 # Uruchomienie klienta
54 def laduj(HOST, PORT):
55
56     # Ustawienie połączenia
57     c = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
58     c.connect((HOST, PORT))
59     c.send('Potwierdzam'.encode())
60
61     t = threading.Thread(target=take_of, args=(c,))
62     t.daemon = True
63     t.start()
64
65     try:
66         while True:
67             msg = input("Napisz wiadomość (lub 'exit' aby zakończyć): ")
68             if msg.lower() == "exit":
69                 break
70             elif msg == "rsa":
71                 # Wywołanie funkcji proszącej o klucz RSA (socket)
72                 prosba(c)
73
74             elif msg.startswith("ENC:"):
75                 print("Wysłano szyfrem")
76                 encrypt(c, msg)
77
78             else:
79                 c.send(msg.encode())
80
81     except KeyboardInterrupt:
82         pass
83     finally:
84         c.close()
85         print("Połączenie zakończone.")
86
```

Funkcja prosba() wysyła do serwera komunikat „GENERUJ_RSA”, co w efekcie wysyła żądanie ustalenia kluczy. Serwer zwraca parametry a klient parsuje je z formatu Json do globalnych zmiennych liczbowych. Obsługiwane są także wyjątki w komunikacji.

```
8 # Żądanie/odbiór klucza publicznego
9 def prosba(c):
10     global global_n, global_e
11     try:
12         # Wysyłanie żądania
13         c.sendall("GENERUJ_RSA".encode())
14
15         # Odbieranie odpowiedzi i weryfikacja
16         data = c.recv(4096)
17         if not data:
18             print("Brak danych od serwera.")
19             return
20         text = data.decode()
21
22         # Próba zdekodowania informacji JSON + obsługa błędów
23         try:
24             rsa_data = json.loads(text)
25             ok = rsa_data.get("status")
26             global_n = int(rsa_data.get("A") or rsa_data.get("n"))
27             global_e = int(rsa_data.get("B") or rsa_data.get("e"))
28             print(f"\n👉 Otrzymano klucz publiczny RSA:\nn= {global_n}\ne = {global_e}\n")
29
30         except json.JSONDecodeError:
31             print("Otrzymano niepoprawny format danych:", data.decode())
32     except Exception as err:
33         print("Błąd przy żądaniu klucza RSA:", err)
34     # return n, e
35
```

Funkcja encrypt() weryfikuje posiadanie klucza a jeśli jest, wykonuje szyfrowanie tekstu (plaintext). Usuwany jest prefix „ENC:” – 4 pierwsze znaki, typ rzutowany jest na liczbę int, i wykonywane jest potęgowanie (pow), które jest wbudowane w Python. W efekcie otrzymujemy zaszyfrowany komunikat i wysyłamy go na serwer.

```
37 # Szyfrowanie wiadomości
38 def encrypt(c, msg):
39     global global_n, global_e
40     if global_n is None or global_e is None:
41         print("❌ Najpierw pobierz klucz RSA komendą: rsa")
42
43     else:
44
45         plaintext = msg[4:] # usuń prefiks ENC:
46         m = int.from_bytes(plaintext.encode("latin-1"), "big")
47         w = pow(m, global_e, global_n)
48         w = str(w)
49         print(w)
50         c.send(f"ENC:{w}".encode())
51
```

Funkcja `take_of( )` utrzymuje połączenie w osobnym wątku, gotowym do pobierania komunikatów z serwera. Komunikaty te są nie szyfrowane. Szyfrowana komunikacja działa tu tylko w stronę klient – serwer.

```
90 # Obsługa wątku klienta
91 def take_of(c):
92     while True:
93         try:
94             data = c.recv(4096)
95             if not data:
96                 print("Klient zakończył połączenie.")
97                 break
98             print(f"\nZ SERWERA - wiadomość nieszyfrowana: {data.decode()}")
99         except ConnectionResetError:
100             print("Połączenie z klientem przerwane.")
101             break
102         except Exception as e:
103             print(f"Błąd odbierania: {e}")
104             break
105
```

5. Podsumowanie

W punktach 3 i 4 opisano strukturę programów a pliki Pythona (py) zostały dodane jako załączniki do projektu. Projekt pozwala na przesyłanie jawnego tekstu pomiędzy serwerem a klientem, co jest implementowane zaraz po uruchomieniu obu. Aby uruchomić proces „negocjacji” klient musi poprosić o przesłanie klucza, zostaje on wyznaczony przez serwer. Jako wynik zwraca klucz publiczny (e, n).

Długość informacji do zaszyfrowania jest zdeterminowana przez długość modułu klucza ‘n’ a więc przez to u źródła także ‘p’ i ‘q’. Generalnie im dłuższy tekst/liczba tym dłuższy wymagany klucz. W przykładzie z załączonym kodem klucz rzędu 10^8 pozwala na zaszyfrowanie 10 znaków. Taki dobór jest czysto edukacyjny, gdyż w praktyce stosuje się klucze 2048 bitowe, co daje nam liczbę rzędu około 10^{617} .

Literatura pokazuje natomiast, że nawet tak dobrany klucz nie jest obecnie bezpieczny i nie jest odporny na ataki. Jako wsparcie bezpieczeństwa stosuje się tzw. padding. Dodaje on ekstra losowe bajty, znaczniki lub przekształcenia matematyczne.

Interakcje między oboma aplikacjami dzieją się poprzez użycie konsoli. Ważna jest tu kolejność uruchamiania – najpierw serwer, później klient. Ważny jest także odpowiedni dobór gniazda. Jak wspomniano, aby przelać zaszyfrowane informacje, klient musi wystawić żądanie klucza RSA (wpisać na konsoli klienta „rsa”) oraz następnie wysyłany tekst do zaszyfrowania poprzedzić prefiksem „ENC:”. Szyfrowanie jest tylko jednostronne, tzn. z klienta do serwera. Bowiem w praktyce RSA jest często używane do negocjacji symetrycznych kluczy innych protokołów. Wystarczy wtedy zatem jednostronna autoryzacja, która da bezpieczeństwo.